COA unit-4

Input–Output Organisation

Input–Output organisation is the method used to transfer data between the computer system and external devices like keyboard, mouse, printer, and monitor. It manages communication between CPU, memory, and I/O devices.

1. I/O Device Interface

An I/O device interface is the hardware that acts as a link between the CPU and input/output devices. Since CPU and I/O devices work at different speeds and use different data formats, the interface helps coordinate communication.

Functions of I/O interface:

- Provides communication between CPU and device
- Controls data transfer
- Converts signals and formats if required
- Stores status and control information

Main components:

- Data Register: Stores data being transferred
- Status Register: Indicates device status (busy/ready)
- Control Register: Sends control commands

Example: Keyboard controller connecting keyboard with CPU.

2. I/O Transfer – Program Controlled I/O

Program Controlled I/O is a data transfer method where the CPU directly controls all I/O operations. The processor continuously checks the status of the device and transfers data when the device becomes ready.

Working:

1. CPU sends command to I/O device
2. CPU checks device status repeatedly
3. When device becomes ready, data transfer occurs
4. CPU continues execution

Advantages: Simple and easy to implement Requires less hardware

Disadvantages: CPU wastes time waiting for device response Reduces overall system performance

Example: CPU continuously checking a printer status before sending data.

Direct Memory Access (DMA)

Direct Memory Access (DMA) is a process of bypassing the processor during data transfer. In software-controlled data transfer, the processor executes a series of instructions for data transfer, and each instruction requires fetch, decode, and execute phases.

The flow of transferring data from memory to an I/O device involves:

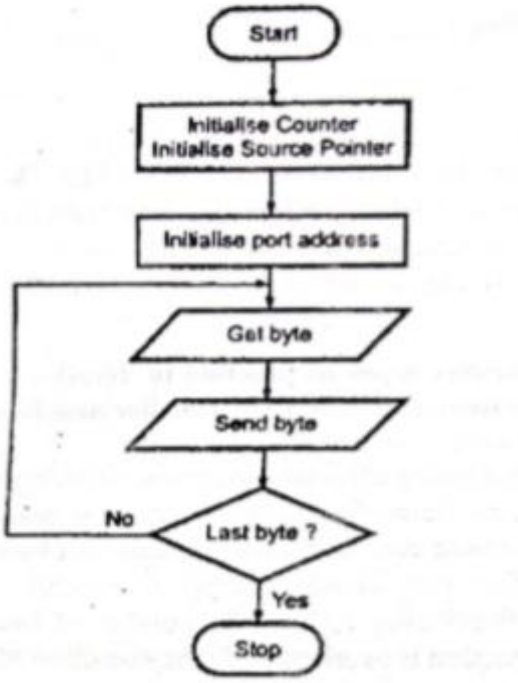
1. Initialize counter and source pointer
2. Initialize port address
3. Get byte
4. Send byte
5. Check whether it is the last byte
6. Repeat until all bytes are transferred

This process consumes considerable processor time. Therefore, software-controlled transfer is not suitable for large data transfers, such as transferring data from a magnetic disk or optical disk to memory.

DMA uses a hardware-controlled data transfer technique, allowing data to move directly between memory and I/O devices without continuous CPU involvement.

Drawbacks of programmed and interrupt-driven I/O:

1. I/O transfer rate is limited by the speed at which CPU can test and service a device.
2. CPU spends too much time checking device status and executing I/O instructions.
To overcome these drawbacks, DMA is used as an alternative method.



Interrupt Driven I/O

Interrupt-driven I/O is a data transfer method in which the CPU does not continuously check the status of an I/O device. Instead, the CPU executes other tasks and the device sends an interrupt signal when it is ready for data transfer. After receiving the interrupt, the CPU temporarily stops its current task, services the I/O request, and then resumes execution.
Working:
1. CPU sends command to I/O device
2. CPU continues other processing work
3. Device generates an interrupt when ready
4. CPU executes Interrupt Service Routine (ISR)
5. CPU resumes previous task
Advantages:
Better CPU utilization
Reduces waiting time
Faster than programmed I/O
Disadvantage:
Interrupt handling adds some overhead

Software Interrupt

A software interrupt is generated by a program or instruction during execution. It is created intentionally by software to request operating system services or handle exceptions.
Examples: System calls Divide-by-zero exception INT instruction in assembly language

Hardware Interrupt

A hardware interrupt is generated by external hardware devices to request CPU attention.
Examples of devices generating hardware interrupts:
Keyboard Mouse Printer Disk controller
Example: When a key is pressed on the keyboard, it sends an interrupt signal to the CPU.

Q.8.(b) Compare Hardware and Software Interrupts. (7)

Ans.

Sr. No.	Hardware Interrupt	Software Interrupt
1.	Hardware interrupt is an interrupt generated from an external device or hardware.	Software interrupt is the interrupt that is generated by any internal system of the computer.
2.	It do not increment the program counter.	It increment the program counter.
3.	Hardware interrupt can be invoked with some external device such as request to start an I/O or occurrence of a hardware failure.	Software interrupt can be invoked with the help of INT instruction.
4.	It has lowest priority than software interrupts.	It has highest priority among all interrupts.
5.	Hardware interrupt is triggered by external hardware and is considered one of the ways to communicate with the outside peripherals, hardware.	Software interrupt is triggered by software and considered one of the ways to communicate with kernel or to trigger system calls, especially during error or exception handling.
6.	It is an asynchronous event.	It is synchronous event.
7.	Hardware interrupts can be classified into two types they are : 1. Maskable Interrupt. 2. Non Maskable Interrupt.	Software interrupts can be classified into two types they are : 1. Normal Interrupts. 2. Exception.
8.	Keystroke depressions and mouse movements are examples of hardware interrupt.	All system calls are examples of software interrupts.

Privileged and Non-Privileged Instructions

Privileged Instructions

Privileged instructions are the instructions that are executed only in kernel mode. If a privileged instruction is attempted in user mode, it is treated as an illegal instruction and is trapped by the operating system through hardware.

It is the responsibility of the operating system to ensure that the timer is set to interrupt before transferring control to any user application. This allows the operating system to regain control when an interrupt occurs. The operating system uses privileged instructions to ensure proper system operation.

Examples of privileged instructions:

1. I/O instructions
2. Context switching
3. Clear memory
4. Set the timer of CPU
5. Halt instructions
6. Interrupt management
7. Modify entries in the device-status table

Non-Privileged Instructions

Non-privileged instructions are the instructions that are executed only in user mode.

Examples of non-privileged instructions:

1. Generate trap instruction
2. Reading system time

3. Reading status of processor
4. Sending output to the printer
5. Performing arithmetic operations

Memory Hierarchy

In a computer system, memory stores program instructions, data values, and intermediate results of calculations. Information in memory is stored in fixed-size cells called bytes. A byte can store a small amount of information such as a character or a numeric value. CPU performs operations on groups of bytes depending on the requirement. The hierarchical arrangement of memories in computer architecture is called Memory Hierarchy.

Memory hierarchy is designed to take advantage of memory locality in computer programs. Each higher level of hierarchy has higher bandwidth, lower latency, and smaller size, while lower levels have larger size but slower access speed. Due to memory transfer delays, the CPU may spend time waiting for memory I/O operations to complete.

Levels of Memory Hierarchy

1. Processor Registers
    - Fastest possible access (usually 1 CPU cycle)
    - Size is only a few hundred bytes
2. Level 1 (L1) Cache
    - Accessed in a few cycles
    - Usually tens of kilobytes
3. Level 2 (L2) Cache
    - Higher latency than L1 by 2× to 10×
    - Usually 512 KB or more
4. Main Memory (DRAM)
    - May take hundreds of cycles
    - Capacity can be multiple gigabytes
5. Disk Storage
    - Millions of cycles latency
    - Very large storage capacity
6. Tertiary Storage
    - Several seconds of latency
    - Extremely large storage size

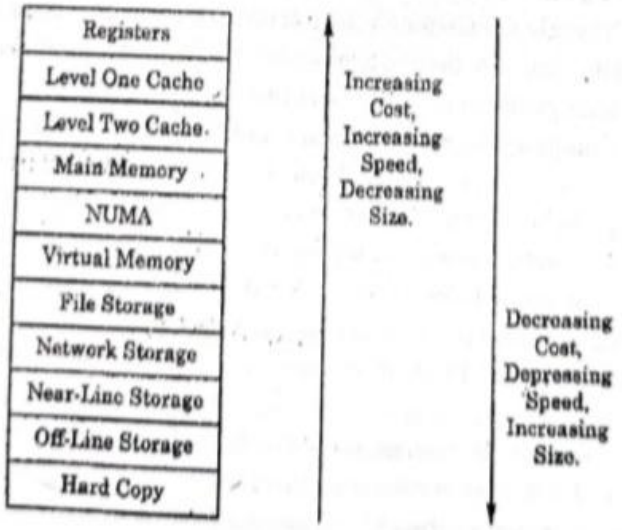


Fig. : A possible form of memory hierarchy

Main Memory

Main memory is the primary memory of a computer used to store programs, data, and instructions currently being processed by the CPU. It is directly accessible by the processor.

Characteristics:

- Faster than secondary storage
- Temporary (volatile in case of RAM)
- Stores active programs and data

Example: RAM (Random Access Memory)

Auxiliary Memory

Auxiliary memory is secondary storage used to store data permanently for future use. It is not directly accessed by the CPU.

Characteristics:

- Large storage capacity

- Non-volatile
- Slower than main memory

Examples: Hard disk, SSD, CD, DVD, Pen drive

Associative Memory

Associative memory, also called Content Addressable Memory (CAM), accesses data by its content rather than memory address.

Characteristics:

- Searches data quickly
- Compares stored content simultaneously
- Used where fast searching is required

Applications: Cache memory and lookup tables

Cache Memory

Cache memory is a small, high-speed memory placed between the CPU and main memory. It stores frequently used instructions and data to reduce access time.

Characteristics:

- Very fast access speed
- Smaller size
- Improves CPU performance

Levels: L1 Cache, L2 Cache, L3 Cache

Virtual Memory

Virtual memory is a memory management technique that allows a system to use a part of secondary storage as an extension of main memory.

Characteristics:

- Allows execution of larger programs than RAM size
- Increases memory capacity virtually
- Slower than actual RAM because disk access is involved

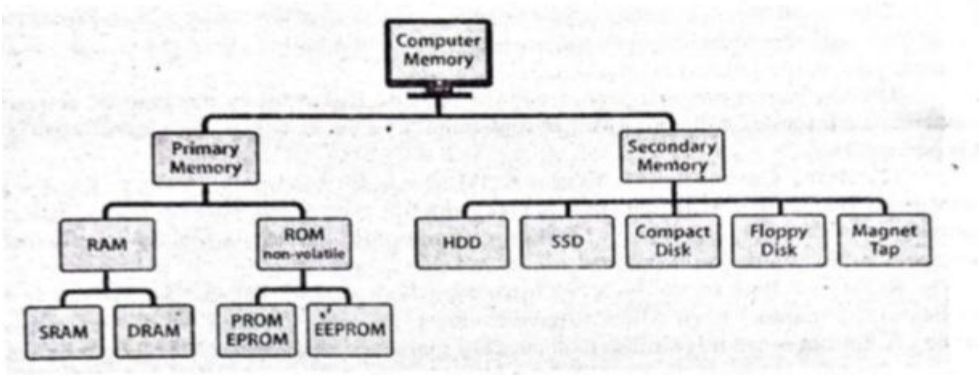
Example: Hard disk space used as swap memory.

Types of Computer Memory

Computer memory is a generic term for different types of data storage technology used by a computer system, including RAM, ROM, and flash memory. Some memories are designed for high speed, while others provide large storage capacity at low cost.

The basic classification of computer memory is:

1. Primary Memory 2. Secondary Memory



The key difference between them is speed of access.

1. Primary Memory

Primary memory includes RAM and ROM and is located close to the CPU. It stores data that the CPU requires immediately so that it can be accessed quickly.

Types of Primary Memory:

(a) RAM (Random Access Memory)

RAM is a memory where data can be accessed in any random order. It is very fast, readable, and volatile, meaning data is lost when power is switched off.

Data required for immediate processing is moved to RAM so the CPU is not kept waiting.

Types of RAM:

(i) DRAM (Dynamic RAM)

- Most common type of RAM
- Stores data in capacitors and transistors
- Requires continuous refreshing to maintain data
- Comes in versions like DDR2, DDR3, and DDR4
- Slower than SRAM

(ii) SRAM (Static RAM)

- Faster than DRAM (2–3 times faster)
- More expensive and bulkier
- Uses six transistors in each cell
- Does not require constant refreshing
- Used as cache memory in CPU

Difference between DRAM and SRAM:

- SRAM is faster than DRAM
- DRAM requires refreshing; SRAM does not
- SRAM is costlier and available in smaller sizes

(b) ROM (Read Only Memory)

ROM stands for Read-Only Memory. Data stored in ROM can be read but normally cannot be written. It is non-volatile, meaning data remains stored even when power is off.

ROM contains bootstrap code, which helps the computer load the operating system during startup.

Types of ROM:

(i) PROM (Programmable ROM)

- Manufactured empty and programmed later using a PROM programmer

(ii) EPROM (Erasable Programmable ROM)

- Data can be erased and reprogrammed
- Erased using ultraviolet light

(iii) EEPROM (Electrically Erasable Programmable ROM)

- Can be erased and rewritten electrically
- Commonly used for firmware and BIOS updates

2. Secondary Memory

Secondary memory is used for long-term storage. It is cheaper per GB but slower than primary memory.

Types of Secondary Memory:

1. Hard Disk Drives (HDD)
2. Solid State Drives (SSD)
3. Optical Drives (CD/DVD)
4. Tape Drives

Secondary memory provides large storage capacity and retains data permanently.

Cache Mapping

Cache mapping is the process of placing blocks of main memory into cache memory. It determines how memory data is mapped from main memory to cache locations.

There are three types of cache mapping:

1. Direct Mapping
2. Associative Mapping
3. Set Associative Mapping

1. Direct Mapping

In Direct Mapping, each block of main memory can be placed in only one fixed location in cache memory.

Mapping function: Cache Line = (Main Memory Block) mod (Number of Cache Lines)

Advantages: Simple and easy to implement Fast access

Disadvantages: High chance of cache conflicts Same cache line may be overwritten frequently

2. Associative Mapping

In Associative Mapping, any block of main memory can be stored in any cache line. The cache checks all entries simultaneously to find the required block.

Advantages: Flexible placement Reduces mapping conflicts

Disadvantages: Hardware is complex More expensive

3. Set Associative Mapping

Set Associative Mapping is a combination of Direct Mapping and Associative Mapping. Cache memory is divided into sets, and a block can be placed in any location within a particular set.

Mapping function: Set Number = (Main Memory Block) mod (Number of Sets)

Advantages: Fewer conflicts than direct mapping Less expensive than associative mapping

Disadvantages:

More complex than direct mapping

This method provides a balance between speed and flexibility.

Writing into Cache (Cache Write Operation)

Writing into cache refers to the process of updating data in cache memory when the CPU performs a write operation. Since data may exist in both cache and main memory, special techniques are used to maintain consistency.

There are two common cache writing methods:

1. Write Through
   - Data is written into cache and main memory simultaneously.
   - Ensures consistency between cache and memory.
   - Simple to implement but slower because main memory is updated every time.
2. Write Back
   - Data is written only into cache memory initially.
   - Main memory is updated later when the cache block is replaced.
   - Faster than write-through but more complex.

Cache Initialization

Cache initialization is the process of preparing cache memory before it begins operation. Initially, the cache does not contain valid data, so it must be set up properly.

Steps in cache initialization:

1. Clear all cache entries
2. Set valid bits to 0 (invalid state)
3. Initialize tag fields
4. Make cache ready for loading data from main memory

After initialization, data blocks from main memory are loaded into cache whenever required.

Purpose of cache initialization:

Removes garbage or old data Ensures correct cache operation Prevents incorrect data access by CPU